Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

Design and Analysis of Algorithms — Assignment 1 (CO1)

Student name	T. Srija
Registration number	192525223
Programme	B.Tech Computer Science and Engineering, SIMATS
Random seed (last 4 digits of Reg No)	5223
Algorithms studied	Linear search (iterative) • Binary search (recursive) • Hash table with open addressing / linear probing (third strategy)
Language / tools	Python 3, matplotlib; all timings by time.perf_counter()
Source code	Python files of the src/ folder submitted with this report (listed in Task 8)

Contents. Task 1 Design • 2 Basic operation • 3 Time-complexity derivation • 4 Best/average/worst case • 5 Asymptotic analysis • 6 Recurrence formulation • 7 Recurrence solution • 8 Implementation & testing • 9 Experimental performance • 10 Theory vs experiment • 11 Final recommendation • 12 Day-4 design challenge (A–D).

Reproducibility. Every number, table and graph in this report was produced by the programs in the src/ folder (benchmark.py, task12.py, make_graphs.py, test_correctness.py, recurrence_check.py). All terminal screenshots show the registration number, name and system date/time printed by the programs themselves; the data come from the seed 5223 (last four digits of 192525223).

Task 1 — Design the Algorithm

Problem. A university keeps one record per student and a registration number (a 9-digit integer) is the search key. For a given key the system must say whether a record exists and where it is stored. The records are not ordered and n grows from 10^3 to 10^6 and beyond. I selected three strategies that are based on three different ideas: *examine every record*, *keep cutting a sorted collection in half* and *calculate where the record has to be*.

	Algorithm 1	**Algorithm 2**	**Algorithm 3**
Name	Linear (sequential) search	Binary search	Hash table, open addressing with linear probing
Design strategy	Brute force	Decrease-and-conquer (discard half each step)	Space–time trade-off (hashing)
Style	Iterative	Recursive	Iterative (build loop + probe loop)
Preprocessing	None — works on the raw, unordered records	Sort all records once: Θ(n log n)	Build the table once: n insertions, Θ(n) expected
Extra memory	None	A sorted list of references	m ≈ 2n slots holding positions

1.1 Algorithm 1 — Linear search (iterative)

Idea. Walk through the records from the first one and stop at the first record whose key equals the search key; reaching the end means that the student is not in the database. **Why selected:** it is the simplest correct method, needs no preparation of the data and therefore is the yardstick that the other two methods must beat.

```
LinearSearch(A[0..n-1], key)
    for i <- 0 to n-1 do
        if A[i] = key then return i        // basic operation
    return -1                              // key not present
```

1.2 Algorithm 2 — Binary search (recursive)

Idea. In a sorted list the middle record tells in which half the key can still lie, so after one comparison half of the remaining records are thrown away. **Preprocessing:** the records are sorted once. **Why selected:** it is the classical way of using order, and its recursive form yields the recurrence relation that is solved in Tasks 6 and 7. (An iterative twin exists in the code only to cross-check the recursive one; it is not counted as a fourth algorithm.)

```
Preprocess: S <- sort(A)                              // Theta(n log n), once

BinarySearch(S, key, low, high)
    if low > high then return -1                       // empty range: not present
    mid <- floor((low + high) / 2)
    if S[mid] = key then return mid                    // basic operation
    if S[mid] < key then return BinarySearch(S, key, mid+1, high)
    else                 return BinarySearch(S, key, low, mid-1)
```

1.3 Algorithm 3 — Hash table with open addressing (linear probing)

Idea. Instead of searching, compute the place: the hash function h(k) = k mod m gives a start slot; if that slot holds another key, the search simply continues with the next slot until the key or an empty slot is met. All records live inside one array (no linked chains). m is the smallest prime ≥ 2n, so the load factor α = n/m stays ≤ 0.5. **Preprocessing:** insert the n records once. **Why selected:** it turns searching into a few array look-ups, which is the right trade-off when searches are very frequent (Task 12).

```
Preprocess: m <- smallest prime >= 2n ; T[0..m-1] <- EMPTY
            for each record at position p with key k:
                i <- k mod m
                while T[i] != EMPTY do i <- (i+1) mod m      // collision: next slot
                T[i] <- p

HashSearch(T, key)
    i <- key mod m
    while T[i] != EMPTY do
        if A[T[i]] = key then return T[i]                   // basic operation
        i <- (i+1) mod m
    return -1
```

Why the three are really different. Linear search uses no knowledge about the data; binary search relies on *order* (and pays for it with a sort); the hash table relies on *arithmetic* on the key (and pays for it with memory and a build phase). Hence they differ in preprocessing, in memory and — as the experiments show — in the way their cost grows with n.

Task 2 — Identify the Basic Operation

In a search the work that decides the running time is the test “is this stored record the wanted one?”. For all three algorithms the **basic operation is the comparison of a stored registration number with the search key**. Index updates, the computation of mid or of k mod m happen a constant number of times per comparison, so they never dominate. Counting comparisons also gives results that do not depend on the computer or on the programming language.

Algorithm	Basic operation and where it occurs	Why appropriate	How the count changes with n
Linear	A[i] == key in the body of the for-loop, once per iteration	The loop does nothing else, so time is proportional to the number of comparisons.	Proportional to n: up to n comparisons, about n/2 on average for existing keys.
Binary (recursive)	S[mid] == key, once in every recursive call	A call performs one comparison plus constant work, so total time ∝ number of calls.	Every comparison halves the range, so the count grows by 1 each time n doubles: about $\log_2 n$.
Hash (linear probing)	A[T[i]] == key for every occupied slot visited while probing	Computing the start slot is one operation; the repeated work is the probe loop.	Depends on the load factor α, not on n: about ½(1 + 1/(1−α)) = 1.5 for α = 0.5 at every n.

File Edit Selection View - algo1_linear.py - Editor

algo1_linear.py

```
1  """Algorithm 1 - linear (sequential) search, iterative."""
2
3
4  def linear_search(a, key):
5      """Return (position or -1, number of key comparisons)."""
6      comps = 0
7      for i in range(len(a)):
8          comps += 1
9          if a[i] == key:              # basic operation: key comparison
10             return i, comps
11     return -1, comps
```

Python | UTF-8

Figure 2.1: Basic operation (A[i] == key, highlighted) in linear search

File Edit Selection View - algo2_binary.py - Editor

algo2_binary.py

```
4  def binary_search(s, key, low=0, high=None, comps=0):
5      """s must be sorted.  Return (position or -1, number of key comparisons)."""
6      if high is None:
7          high = len(s) - 1
8      if low > high:                    # base case: empty range
9          return -1, comps
10     mid = (low + high) // 2
11     comps += 1
12     if s[mid] == key:                 # basic operation: key comparison
13         return mid, comps
14     if s[mid] < key:
15         return binary_search(s, key, mid + 1, high, comps)
16     return binary_search(s, key, low, mid - 1, comps)
```

Python | UTF-8

Figure 2.2: Basic operation (s[mid] == key, highlighted) in recursive binary search

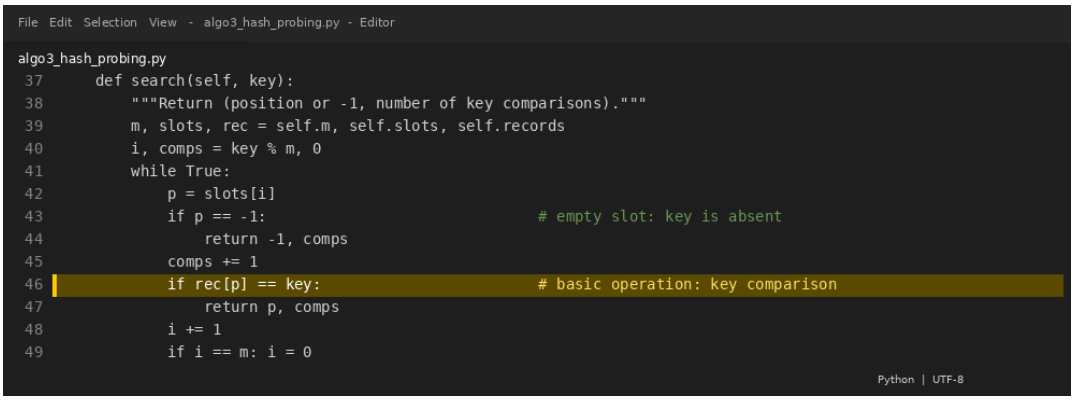


Figure 2.3: Basic operation (rec[p] == key, highlighted) in the probe loop of the hash table

Task 3 — Derive the Time-Complexity Function

Input size: n = number of student records. Counted operation: key comparisons C(n); q is the number of searches performed after the preprocessing.

3.1 Linear search

Worst case. The key is in the last position or absent; the loop body runs for i = 0, …, n−1 and compares once each time, so $C_{worst}(n) = n$.

Average case (key present). If every position is equally likely, a key at position i (counting from 1) costs i comparisons: $C_{avg}(n) = (1/n)(1 + 2 + \ldots + n) = (n + 1)/2$. If a fraction p of the queries are successful, $C_{avg} = p(n+1)/2 + (1-p)n$.

Preprocessing P(n) = 0. For q searches the expected cost is q(n+1)/2; for $n = 10^6$ this is 500,000.5 comparisons per search.

3.2 Binary search

Worst case. One call keeps at most ⌊n/2⌋ records, so C(n) = C(⌊n/2⌋) + 1 with C(1) = 1 (solved in Task 7), i.e. $C_{worst}(n) = \lfloor \log_2 n \rfloor + 1$. For $n = 10^6$ this is ⌊19.93⌋ + 1 = 20.

Average case (key present). Take $n = 2^k - 1$. The decision tree has 2^{d-1} keys on level d (d = 1, …, k) and a key on level d is found after d comparisons, so $C_{avg} = (1/n)\ \Sigma\ d \cdot 2^{d-1} = ((k-1)2^k + 1)/n = (k - 1) + k/n \approx \log_2(n+1) - 1$. For $n = 10^6$ this gives ≈ 18.93.

Preprocessing. Sorting by comparisons needs Θ(n log n) steps, about 19,931,569 for $n = 10^6$. Total for q searches: $T(n, q) = n \log_2 n + q \log_2 n = (n + q) \log_2 n$.

3.3 Hash table with linear probing

Assumption: uniform hashing — every key is equally likely to start in any of the m slots; load factor α = n/m ≤ 0.5. The classical result (Knuth) for an *unsuccessful* search at load x is $U(x) = \frac{1}{2}(1 + 1/(1-x)^2)$ probes, the last probe being the empty slot.

Successful search. A key inserted when the table held i keys needs the same probes that an unsuccessful search needed at that moment, namely U(i/m). Averaging over the n insertions: $C_{succ} \approx (1/\alpha) \int_0^{\alpha} \frac{1}{2}(1 + 1/(1-x)^2)\ dx = \frac{1}{2}(1 + 1/(1-\alpha))$. For α = 0.5 this is **1.5**, independent of n. **Unsuccessful search.** U(α) = 2.5 probes, but the final probe only inspects an empty slot, so the number of key comparisons is U(α) − 1 = 1.5.

Worst case. If all keys collide the probe sequence has length n, so $C_{worst} = n$; with a good hash function only short sequences occur. **Preprocessing.** n insertions of about 1.5 probes each: P(n) = Θ(n) expected. Total for q searches: $T(n, q) = c_1 n + c_2 q = \Theta(n + q)$.

3.4 Preprocessing and searching together

Algorithm	Preprocessing P(n)	One search (average)	Total for q searches	$n = 10^6$, q = 50,000
Linear	0	(n+1)/2	q(n+1)/2	≈ 25,000,025,000 comparisons
Binary	$n \log_2 n$	$\log_2(n+1) - 1$	$n \log_2 n + q(\log_2(n+1) - 1)$	≈ 20,878,147 comparisons
Hash (probing)	≈ 1.5 n probes	1.5	1.5 n + 1.5 q	≈ 1,575,000 probes + comparisons

Break-even. Binary search repays its sort after q* searches, where $q^* \cdot ((n+1)/2 - (\log_2(n+1) - 1)) = n \log_2 n$, hence $q^* = 39.9 \approx 40$ searches for $n = 10^6$ (about $2 \log_2 n$). In comparison counts the hash table is cheaper than the sort already before the first search ($1.5n$ against $n \log_2 n$), but its build uses more memory and, in real time, about as much time as sorting (Task 9).

Task 4 — Best, Average and Worst-Case Behaviour

4.1 Linear search

Best case — 1 comparison: the wanted registration number is the first record. **Worst case — n comparisons:** the key is the last record or is not stored (in my tests reg no + 1 = 192525224), so the whole list has to be read. **Average case — (n+1)/2:** half of the list is read on average; at $n = 10^6$ I measured 497,581 comparisons for existing keys and exactly 1,000,000 for missing keys. The distance between the cases is so large because the algorithm has no information that allows it to skip a record.

4.2 Binary search

Best case — 1 comparison: the key is the middle record of the sorted list (tested with $S[(n-1)/2]$). **Worst case — $\lfloor \log_2 n \rfloor + 1$:** the key is absent or lies on the deepest level of the decision tree; the measured maxima were 10, 14, 17, 20 for $n = 10^3, \ldots, 10^6$, exactly $\lfloor \log_2 n \rfloor + 1$. **Average case ≈ $\log_2(n+1) - 1$:** measured over *every* stored key it is 8.99, 12.36, 15.69, 18.95; for absent keys 9.97, 13.36, 16.68, 19.95. Because half of all keys sit on the last level, a typical search is nearly as deep as the worst one.

4.3 Hash table

Best case — 1 comparison: the key sits in its home slot. **Average case ≈ 1.5:** measured over every stored key 1.47, 1.53, 1.50, 1.50 comparisons (theory $\frac{1}{2}(1 + 1/(1-\alpha)) = 1.50$); absent keys cost 1.81, 1.49, 1.56, 1.40. **Worst case — the longest probe sequence:** theoretically $\Theta(n)$ if all keys collide; in my data the longest sequence was 15, 21, 30, 49 for the four sizes. Linear probing builds runs of occupied slots (primary clustering), which is why the worst case is much larger than the average of 1.5, but it still grows only slowly because k mod m spreads random 9-digit numbers evenly.

n	Linear: best / avg(found) / worst	Binary: best / avg(found) / worst	Hash: best / avg(found) / worst
1,000	1 / 500.25 / 1,000	1 / 8.99 / 10	1 / 1.47 / 27
10,000	1 / 5,001.32 / 10,000	1 / 12.36 / 14	1 / 1.53 / 21
100,000	1 / 50,006.04 / 100,000	1 / 15.69 / 17	1 / 1.50 / 30
1,000,000	1 / 497,581.23 / 1,000,000	1 / 18.95 / 20	1 / 1.50 / 49

Table 4.1: Measured number of comparisons per case (produced by benchmark.py; results/bench_*.json).

Task 5 — Asymptotic Analysis

Algorithm	Searching	Preprocessing	Notation used and why
Linear	Worst and average: Θ(n); best: Θ(1). The algorithm as a whole is O(n) and Ω(1).	none	Worst and average cases are squeezed between $c_1 n$ and $c_2 n$, so Θ(n) is exact for them; the best case is constant, therefore only O(n) holds for the algorithm in general.
Binary	Worst and average: Θ(log n); best: Θ(1).	Θ(n log n) sort	$\lfloor \log_2 n \rfloor + 1$ is bounded from above and below by constant multiples of log n, hence Θ. With q searches: Θ((n + q) log n).
Hash	Expected Θ(1) while α ≤ 0.5; worst case O(n).	Θ(n) build	The expected count 1.5 is a constant for every n, so Θ(1) is exact for the expected case; O(n) is only an upper bound reached by total collision.

Growth comparison. From slowest to fastest: $1 \prec \log n \prec n$, because $\lim_{n\to\infty} (\log n)/n = 0$ and $\lim_{n\to\infty} 1/\log n = 0$. Therefore for any constant c the linear cost eventually exceeds c·log n, which eventually exceeds every constant.

n	Linear: n/2	Binary: $\log_2 n$	Hash: ½(1+1/(1−α))	Linear / Binary	Linear / Hash
1,000	500	10.0	1.50	50 ×	333 ×
10,000	5,000	13.3	1.50	376 ×	3,333 ×
100,000	50,000	16.6	1.50	3,010 ×	33,333 ×
1,000,000	500,000	19.9	1.50	25,086 ×	333,333 ×

Table 5.1: Predicted comparisons per successful search.

Interpretation for the university database. When the database grows 1,000-fold ($10^3 \to 10^6$ records) linear search becomes 1,000 times more expensive (500 → 500,000 comparisons), binary search needs only 10 more comparisons (10 → 20; one extra comparison whenever the data double) and the hash table does not need a single extra comparison. The measured times point in the same direction: ×2,749 for linear, ×2.4 for binary and ×2.6 for the hash table (Task 10 explains why the last two are not flat). For the preprocessing the order is: none < sort ≈ hash build, so the investment has to be earned back by enough searches (Task 12).

Task 6 — Formulate the Recurrence Relation

Binary search is the recursive algorithm, so the recurrence is read directly from its code (Task 1.2).

```
BinarySearch(S, key, low, high)
    if low > high then return -1                        // (1) base case
    mid <- floor((low + high) / 2)
    if S[mid] = key then return mid                     // (2) one comparison, O(1) work
    if S[mid] < key then return BinarySearch(S, key, mid+1, high)   // (3) ONE call, right part
    else                 return BinarySearch(S, key, low, mid-1)    // (3) ONE call, left part
```

Component	Identification from the algorithm
Input size	n = high − low + 1, the number of records still in play (initially all n).
Size of the sub-problem	The call continues on S[mid+1..high] (size $\lfloor n/2 \rfloor$) or on S[low..mid−1] (size $\lfloor (n-1)/2 \rfloor$). For the worst case the larger part, $\lfloor n/2 \rfloor$, is taken.
Number of recursive calls	Exactly one per invocation — only one of the two branches runs, the other half is discarded.
Work outside the call	Computing mid and testing S[mid] against the key: constant time c = Θ(1), counted as one key comparison.
Base case	n = 0 (low > high) returns −1 without a comparison; n = 1 needs one comparison, so T(1) = 1.

Putting it together. Searching n records costs one constant step plus the cost of searching at most $\lfloor n/2 \rfloor$ records: $T(n) = T(\lfloor n/2 \rfloor) + c$ for $n > 1$, $T(1) = c_0$. Counting key comparisons only ($c = c_0 = 1$): **$C(n) = C(\lfloor n/2 \rfloor) + 1$, $C(1) = 1$**.

Check on a small case. n = 15: the call compares the middle record and continues with at most 7 records, then 3, then 1 — in total at most 1 + 1 + 1 + 1 = 4 = $\lfloor \log_2 15 \rfloor + 1$ comparisons. The program's own counter reports 4 as well (Figure 7.1).

Task 7 — Solve the Recurrence Relation

Recurrence from Task 6: $T(n) = T(\lfloor n/2 \rfloor) + c$, $T(1) = c_0$. It is solved by back-substitution, confirmed with the Master Theorem and proved by induction.

7.1 Iteration (back-substitution) method

The recurrence has one recursive term whose argument is halved, so repeated expansion shows a clear pattern. For $n = 2^k$ the floors disappear:

```
T(n) = T(n/2) + c
     = [T(n/4) + c] + c            = T(n/4) + 2c
     = [T(n/8) + c] + 2c           = T(n/8) + 3c
     ...
     = T(n/2^i) + i*c              (after i expansions)

The expansion stops when n/2^i = 1, i.e. i = log2 n:
T(n) = T(1) + c*log2 n = c0 + c*log2 n        ->  T(n) is Theta(log n)
```

For general n the floors change the number of levels by at most one: $T(n) = c(\lfloor \log_2 n \rfloor + 1) + c_0 - c$, still $\Theta(\log n)$.

7.2 Master Theorem check

$T(n) = aT(n/b) + f(n)$ with $a = 1$, $b = 2$ and $f(n) = c = \Theta(1)$. Then $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$ and $f(n) = \Theta(n^{\log_b a})$, so **Case 2** applies: $T(n) = \Theta(n^{\log_b a} \log n) = \Theta(\log n)$. Both methods agree.

7.3 Substitution (induction) proof

Claim: $T(n) \le c \log_2 n + c_0$ for all $n \ge 1$ (with $c = c_0 = 1$ this is $C(n) \le \log_2 n + 1$). **Base:** $T(1) = c_0 = c \cdot 0 + c_0$. **Step:** assume the claim for n/2. Then $T(n) = T(n/2) + c \le c \log_2(n/2) + c_0 + c = c(\log_2 n - 1) + c_0 + c = c \log_2 n + c_0$. ∎

7.4 What the result tells us

$T(n) = \Theta(\log n)$ means that the number of comparisons — and the depth of the recursion — grows by only a constant whenever n doubles: 10 levels for 1,000 records, 20 for 1,000,000 and only 30 for 10^9 records. The call stack never exceeds O(log n) frames (20 here), so the recursive version is safe even for millions of records. The decisive fact is that every call makes *one* recursive call; with two calls (a = 2) the same recurrence would give $\Theta(n)$ and the whole advantage of halving would be lost.

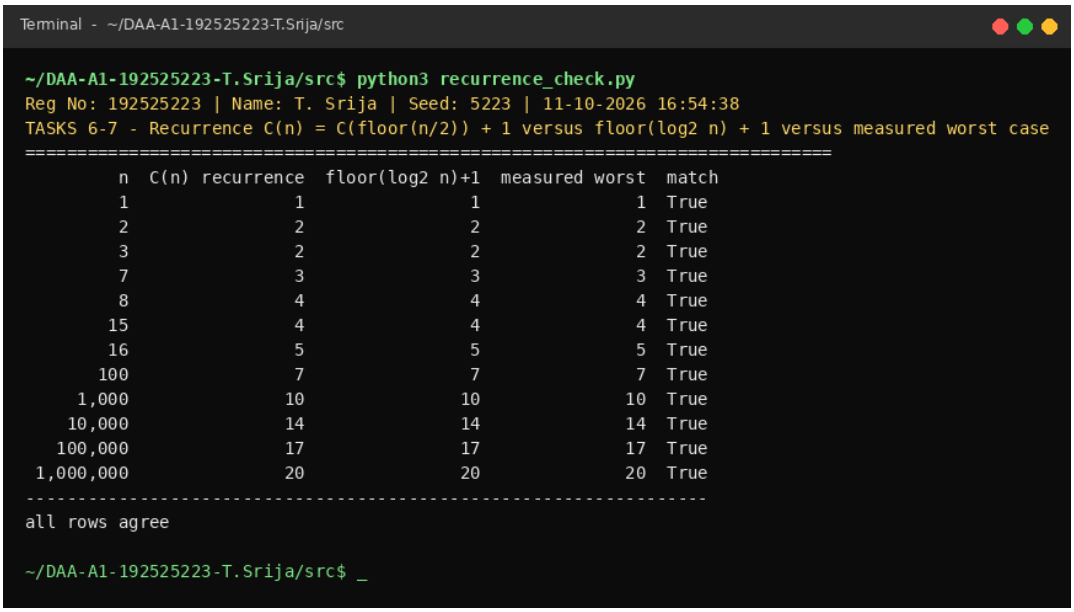


Figure 7.1: C(n) from the recurrence, $\lfloor \log_2 n \rfloor + 1$ and the measured worst case agree for every tested n

Task 8 — Implementation and Testing

Implementation. The three algorithms are written in Python exactly as in Task 1 (src/algo1_linear.py, algo2_binary.py, algo3_hash_probing.py). Each search returns the position (or −1) *and* the number of key comparisons it executed, so one run gives both correctness evidence and operation counts.

Datasets. src/common.py generates n = 1,000, 10,000, 100,000 and 1,000,000 unique 9-digit registration numbers in random order from a generator seeded with 5223 (+ n), the last four digits of 192525223. Each dataset contains my own registration number 192525223 (found test) and never contains 192525224 (not-found test). All three algorithms receive the same records: linear search the raw list, binary search a sorted copy, the hash table is built from the raw list.

Test procedure (src/test_correctness.py). For every size 26 existing keys (own reg no, first and last record of the raw list, smallest, largest and median of the sorted list, 20 random records) and 6 non-existing keys (reg no + 1, 0, below the minimum, above the maximum, 999,999,999 and a number next to the median) are searched in all three structures. Every answer is compared with an independent oracle (a Python dictionary) and the iterative binary search cross-checks the recursive one. Edge cases: empty list and one-record list.

Test case	Expected result	Why included
Own reg no 192525223	found by all three algorithms, correct position	personal key required by the rules
Own reg no + 1 = 192525224	not found	personal key required by the rules
First / last record (raw), min / max (sorted), median	found	boundary positions and the binary-search best case
0, below min, above max, 999,999,999, median + gap	not found	out-of-range and in-range absent keys
20 random existing keys	found	general behaviour
empty list; one-record list	not found / found	degenerate sizes

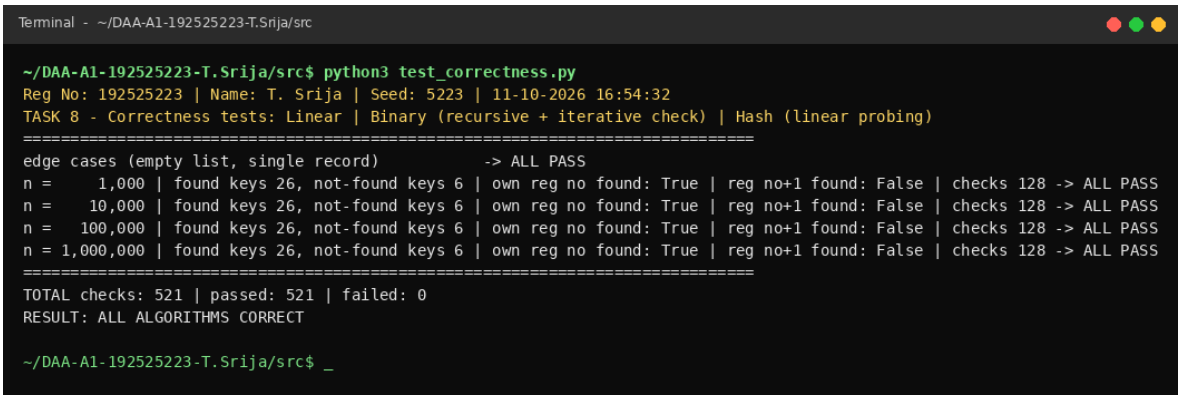


Figure 8.1: Correctness test output for all algorithms at n = 1,000, 10,000, 100,000 and 1,000,000 (521 checks, 0 failures)

Outcome: all 521 checks passed, so the three implementations are correct and are the programs analysed in Tasks 1–7. To reproduce: `cd src && python3 test_correctness.py` (Python 3.8+, no external libraries).

Task 9 — Record Experimental Performance

Method. src/benchmark.py measures for every size the preprocessing time and the average time and number of comparisons per search. The same seeded sample of 1,000 existing keys (the first one is my own reg no, the others are drawn one from each of 999 equal blocks of the list) and 1,000 non-existing keys (the first one is reg no + 1) is used for binary search and hashing; linear search uses 1,000 queries up to n = 10^4, 300 at n = 10^5 and 100 at n = 10^6 (otherwise a single run would take minutes). Preprocessing time is the best of 3 repeats for n ≤ 10^4 and one run for larger n; all times come from time.perf_counter() on the same machine and Python process settings. The average comparison counts of binary search and hashing for existing keys in Tasks 4 and 10 are exact averages over all n stored keys.

Algorithm	Prep. (ms)	Avg time, found (µs)	Avg time, not found (µs)	Avg comps, found	Avg comps, not found	Best	Worst
Linear search	0.00	26.99	64.70	500.25	1,000.00	1	1,000
Binary search (recursive)	0.10	2.43	2.91	8.99	9.97	1	10
Hash (linear probing)	0.17	0.40	0.53	1.47	1.81	1	27

Table 9.1: Results for n = 1,000 (linear search: 1000 queries, binary/hash: 1,000 found + 1,000 not-found keys).

Algorithm	Prep. (ms)	Avg time, found (µs)	Avg time, not found (µs)	Avg comps, found	Avg comps, not found	Best	Worst
Linear search	0.00	235.33	457.71	5,001.32	10,000.00	1	10,000
Binary search (recursive)	1.84	2.28	2.45	12.33	13.36	1	14
Hash (linear probing)	2.20	0.36	0.37	1.53	1.49	1	21

Table 9.2: Results for n = 10,000 (linear search: 1000 queries, binary/hash: 1,000 found + 1,000 not-found keys).

Algorithm	Prep. (ms)	Avg time, found (µs)	Avg time, not found (µs)	Avg comps, found	Avg comps, not found	Best	Worst
Linear search	0.00	3,647.73	6,897.63	50,006.04	100,000.00	1	100,000
Binary search (recursive)	20.87	3.55	3.63	15.71	16.68	1	17
Hash (linear probing)	19.70	0.59	0.61	1.50	1.56	1	30

Table 9.3: Results for n = 100,000 (linear search: 300 queries, binary/hash: 1,000 found + 1,000 not-found keys).

Algorithm	Prep. (ms)	Avg time, found (µs)	Avg time, not found (µs)	Avg comps, found	Avg comps, not found	Best	Worst
Linear search	0.00	74,214.22	154,479.59	497,581.23	1,000,000.00	1	1,000,000
Binary search (recursive)	357.95	5.74	5.84	18.96	19.95	1	20
Hash (linear probing)	409.08	1.03	1.06	1.48	1.40	1	49

Table 9.4: Results for n = 1,000,000 (linear search: 100 queries, binary/hash: 1,000 found + 1,000 not-found keys).

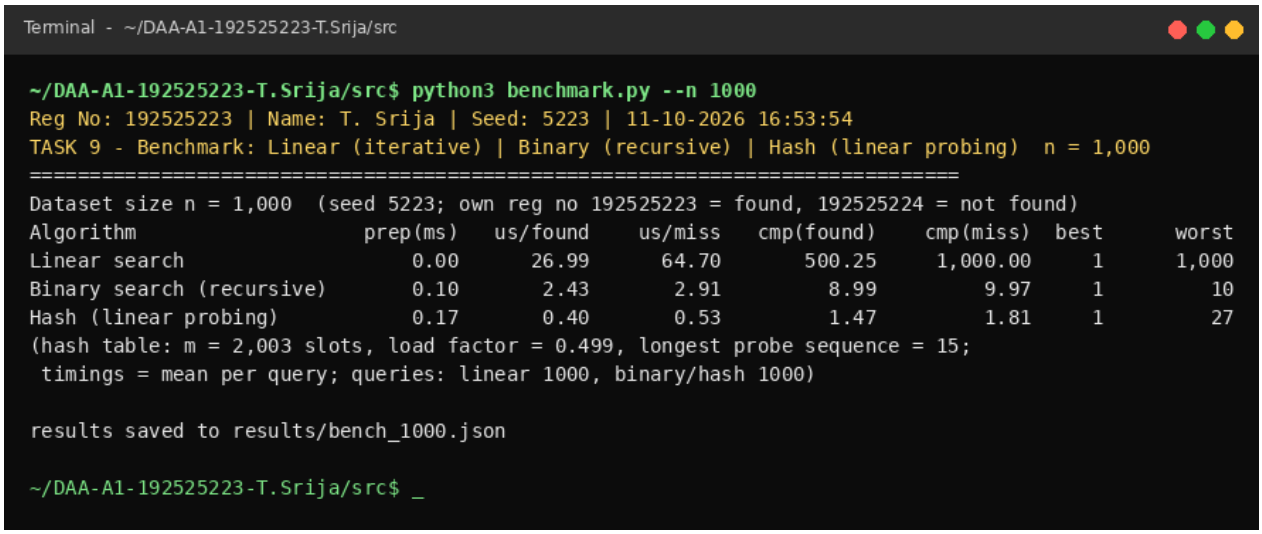


Figure 9.1: Benchmark terminal output for n = 1,000

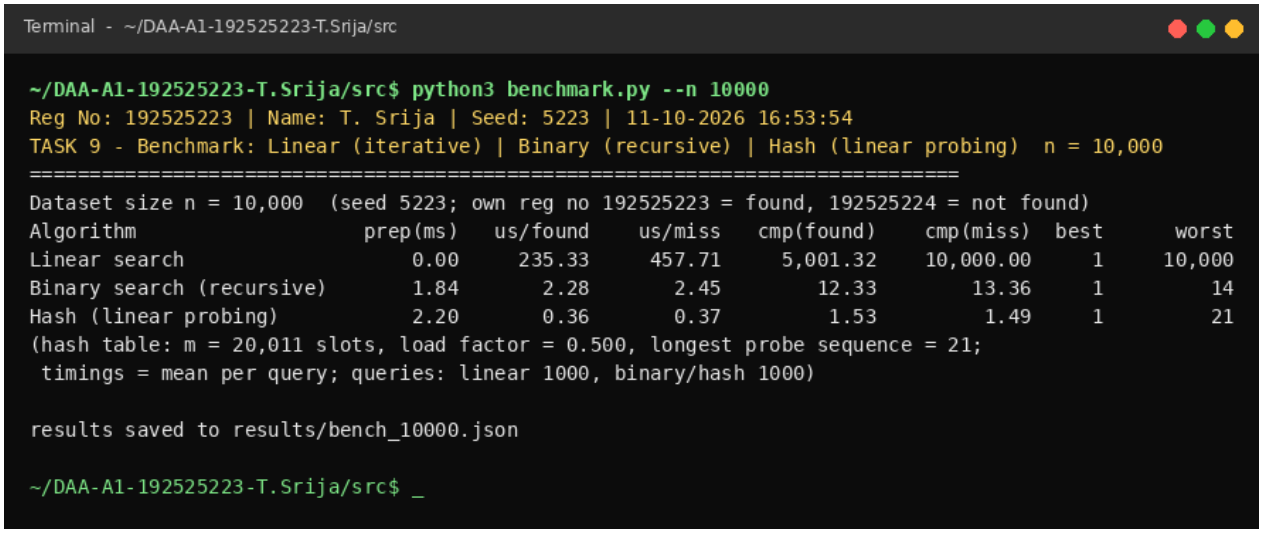


Figure 9.2: Benchmark terminal output for n = 10,000

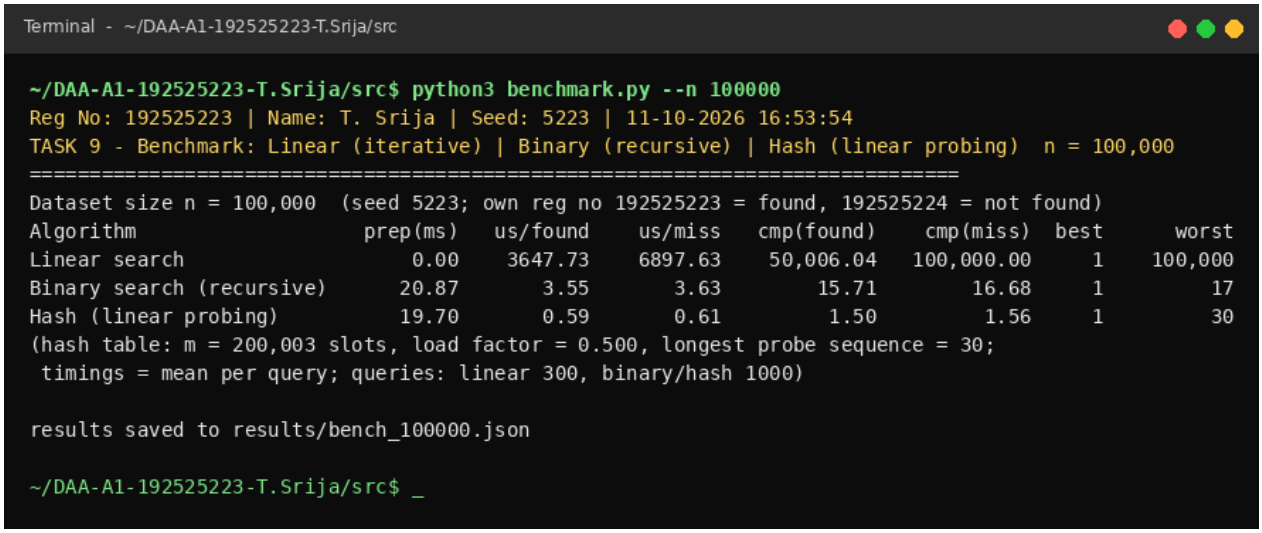


Figure 9.3: Benchmark terminal output for n = 100,000

```
Terminal - ~/DAA-A1-192525223-T.Srija/src

~/DAA-A1-192525223-T.Srija/src$ python3 benchmark.py --n 1000000
Reg No: 192525223 | Name: T. Srija | Seed: 5223 | 11-10-2026 16:53:58
TASK 9 - Benchmark: Linear (iterative) | Binary (recursive) | Hash (linear probing)  n = 1,000,000
================================================================================
Dataset size n = 1,000,000  (seed 5223; own reg no 192525223 = found, 192525224 = not found)
Algorithm                 prep(ms)   us/found    us/miss   cmp(found)    cmp(miss)  best      worst
Linear search                 0.00   74214.22  154479.59   497,581.23 1,000,000.00     1  1,000,000
Binary search (recursive)     357.95      5.74       5.84        18.96        19.95     1         20
Hash (linear probing)         409.08      1.03       1.06         1.48         1.40     1         49
(hash table: m = 2,000,003 slots, load factor = 0.500, longest probe sequence = 49;
 timings = mean per query; queries: linear 100, binary/hash 1000)

results saved to results/bench_1000000.json

~/DAA-A1-192525223-T.Srija/src$ _
```

Figure 9.4: Benchmark terminal output for n = 1,000,000

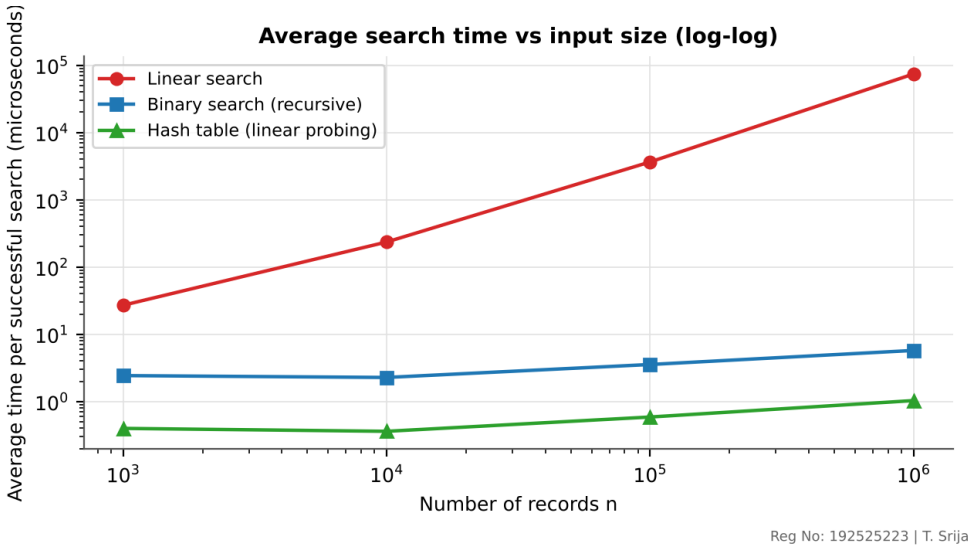


Figure 9.5: Average search time vs n (log-log axes)

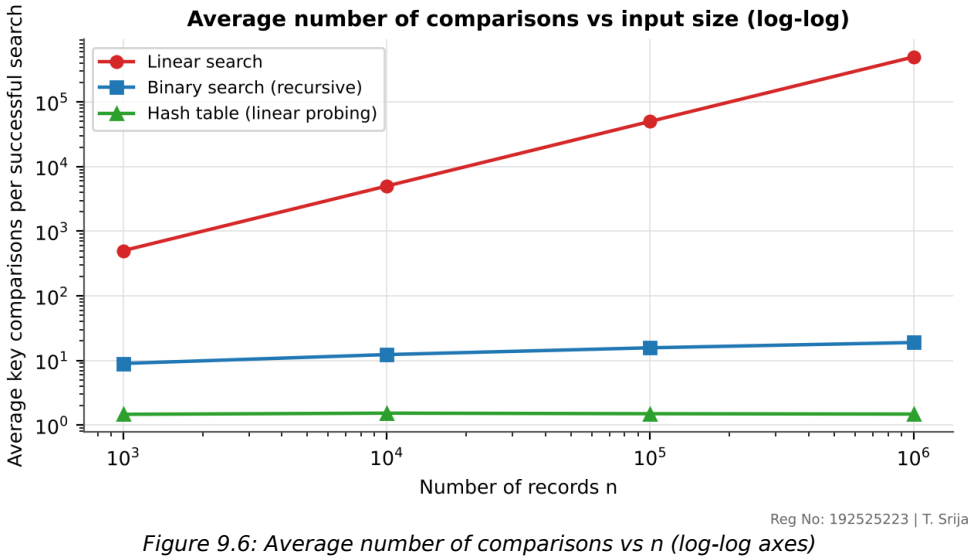


Figure 9.6: Average number of comparisons vs n (log-log axes)

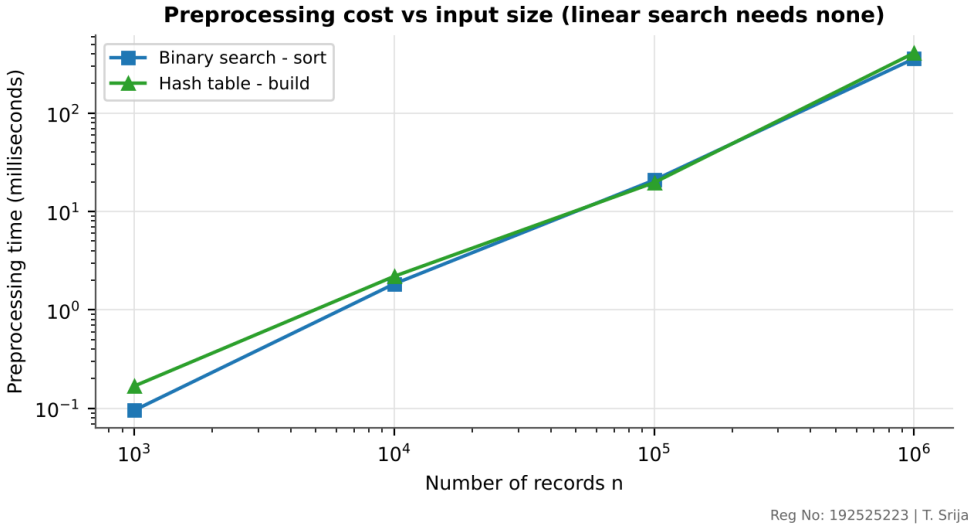


Figure 9.7: Preprocessing time vs n (log-log axes)

Task 10 — Compare Theory with Experimental Behaviour

Comparison counts agree with the derivations. Figure 10.1 plots the measured averages against the formulas of Task 3. At n = 10^6 linear search needed 497,581 comparisons against the predicted 500,000.5 (difference -0.48%, sampling error of 100 queries). Binary search needed 18.95 against the predicted 18.93, and its worst case equals $\lfloor \log_2 n \rfloor + 1$ at every size. The hash table needed 1.50 against $\frac{1}{2}(1 + 1/(1-\alpha)) = 1.50$ (measured over all keys); the right-hand panel is drawn on a magnified axis (1.3–1.7), so the small wiggles are only rounding of values that are all within about 2% of 1.5.

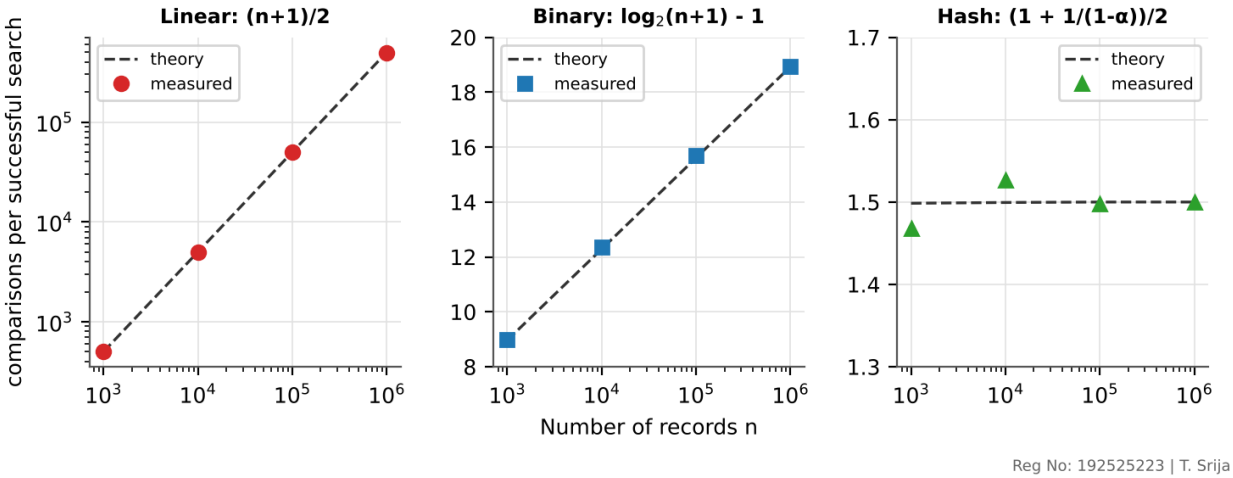


Figure 10.1: Theoretical (dashed) versus measured average comparisons for existing keys

Execution time follows the same order of growth, but not perfectly. Figure 10.2 divides the measured time by the theoretical growth function (n, log n, 1) and normalises the result to n = 1,000; a flat line at 1 would mean perfect agreement.

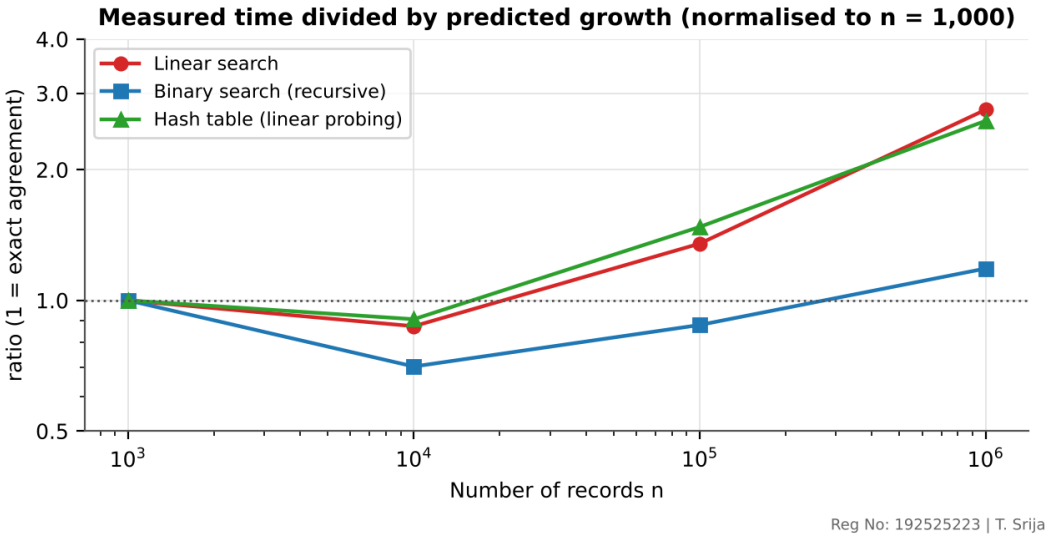


Figure 10.2: Measured time divided by predicted growth (normalised to n = 1,000)

Which scales best? The hash table has the flattest growth, binary search is next and linear search is far behind: at n = 10^6 one linear search costs 12,920× one binary search and 71,937× one hash look-up.

Differences between prediction and measurement, and likely reasons

Observation	Possible reasons
Binary-search time grew ×2.4 from 10^3 to 10^6 records; the model predicts about ×2 (10 → 20 comparisons).	Cache effects: a 1,000-element list fits into the processor cache, a million-element list does not, so the deeper comparisons of each search miss the cache. Python's overhead for about 20 nested calls is a further constant not contained in the comparison model.
Hash-search time grew ×2.6 although the comparison count stayed at 1.5.	The count is constant, but every look-up touches the slot array, then the record list and an int object at addresses that are effectively random. At 10^6 records this memory no longer fits into the cache, so the real cost of "O(1)" depends on the memory hierarchy, which the model ignores.
Time per comparison of linear search rose from 54 ns (n = 10^3) to 149 ns (n = 10^6).	The list holds pointers to separately allocated integers whose order in memory differs from the order in the list, so a sequential scan at 10^6 records also touches memory unpredictably. The growth of the total time (×2,749) therefore exceeds the ×1,000 of the comparisons.
Timings vary from run to run, comparison counts do not.	Time is affected by CPU scheduling, frequency scaling and garbage collection; counts are deterministic for a seed. That is why the theory is validated mainly with counts and the timings are used for the overall trend.
Hash build (409 ms) took a similar time to sorting (358 ms) at n = 10^6, although the theory says Θ(n) against Θ(n log n).	Sorting runs inside Python's compiled sort routine, whereas the hash build is an interpreted loop with one iteration per record. The constant factor, not the growth rate, differs; for much larger n the Θ(n) build would overtake.

Conclusion of the comparison. The mathematical analysis predicts the growth and the ranking of the algorithms correctly. Absolute times differ from the plain operation counts because of constant factors, the memory hierarchy and the implementation language.

Task 11 — Recommend and Defend the Final Design

Recommendation: binary search on a sorted array (recursive or iterative), with the records sorted once as preprocessing.

Why it is more appropriate than the alternatives

- **Versus linear search.** At n = 10^6 a linear search needs 497,581 comparisons and 74.2 ms, binary search needs 19.0 comparisons and 5.7 µs — about 12,920× faster. The one-off sort (358 ms) is repaid after about 5 searches, which a university system performs within seconds.
- **Versus the hash table.** The hash table is faster per search (1.03 µs against 5.74 µs) and costs about the same to build, but it needs 44 MB instead of 8 MB, its worst case is Θ(n) (longest probe sequence 49 here) instead of a guaranteed 20 comparisons, and it can only answer exact-match questions. A sorted array also answers range questions that staff are likely to ask ("all students with registration numbers between X and Y") in O(log n + k).

Criterion (n = 10^6)	Linear	Binary (recommended)	Hash (linear probing)
Theoretical search cost	Θ(n)	Θ(log n)	Θ(1) expected
Measured avg comparisons (found)	497,581.23	18.95	1.50
Measured avg time per search (µs)	74,214.22	5.74	1.03
Preprocessing time (ms)	0	357.95	409.08
Extra memory	none	8.0 MB	44.0 MB
Worst case	n	20	49 measured, Θ(n) theoretical
Range / ordered queries	yes (slow)	yes (fast)	no

Table 11.1: Evidence used for the recommendation (from my own experiments and derivations).

Trade-offs and limitations. (1) The array has to stay sorted: inserting one record into a sorted array costs O(n), so new records should be added in batches and merged (Task 12). (2) The sort is an up-front Θ(n log n) cost — irrelevant when many searches follow, but not worth it for a handful of one-off look-ups, where linear search is simpler. (3) A hash table would be the better choice only if exact-match look-ups clearly dominate and memory is plentiful.

Does it stay practical as the database grows? Yes. Because T(n) = Θ(log n), growing from 10^6 to 10^9 records adds about 10 comparisons per search (20 → 30) and the sort cost grows only slightly faster than linearly, whereas linear search would become 1,000 times slower.

Task 12 — Day-4 Design Challenge: Re-evaluate and Adapt the Design

Changed requirement: 50,000 search requests per hour on 1,000,000 records; new records are added only once per day.

A. Impact Analysis (2 marks)

What has changed. Until now the workload was open: the question was how fast *one* search can be. Now the load is fixed: 50,000 searches per hour = 13.9 searches per second = 1,200,000 searches per day, more searches per day than records in the database, while the data themselves change just once per day.

Assumptions of the original design that are affected. (i) In Tasks 3–11 the preprocessing was an expense that had to be justified against a few searches; with 1.2 million searches a day it is shared by all of them. (ii) A per-search difference that looked tiny is now multiplied by 1.2 million. (iii) The need to support insertions at any moment, which makes sorted arrays and hash tables awkward, disappears.

Why search frequency matters. Total cost = preprocessing + number of searches × cost per search. With a huge number of searches the second term dominates, so the algorithm with the cheapest search wins unless its preprocessing or memory is unaffordable. **Why rare updates matter.** If records arrive once a day, the index can be rebuilt or merged during the night and then stays read-only for 24 hours: no per-insertion cost, no locking and no re-balancing in the daytime. A static structure is exactly what binary search and hashing prefer; their weak point (expensive insertion) becomes irrelevant.

B. Evaluation of the Original Design (2 marks)

Original recommendation: binary search on a sorted array. src/task12.py generated 50,000 queries on 10^6 records (about 90% existing and 10% missing keys, including my reg no and reg no + 1, in random order) and measured:

Design	Prep. (s)	50,000 searches (s)	Per search (µs)	Max searches/s (1 core)	Extra memory (MB)	Cost of one day (s)
Linear (projected from 100 measured searches)	0	4,134.52	82,690.49	12.1	0	99,229
Binary search (original)	0.40	0.56	11.17	89,557	8	13.80
Hash table (linear probing)	0.47	0.06	1.13	881,922	44	1.83

Table 12.1: Benchmark of the three designs for 50,000 searches on 1,000,000 records. 'Cost of one day' = preprocessing + 24 × (time for 50,000 searches).

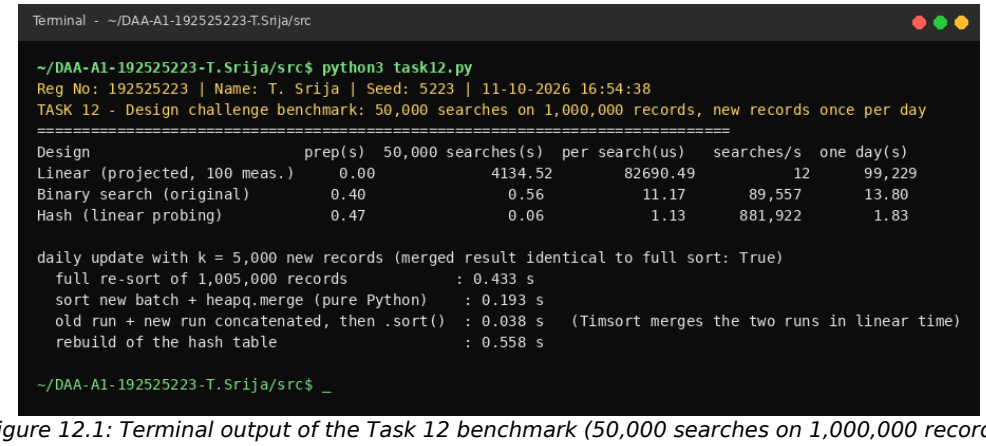


Figure 12.1: Terminal output of the Task 12 benchmark (50,000 searches on 1,000,000 records)

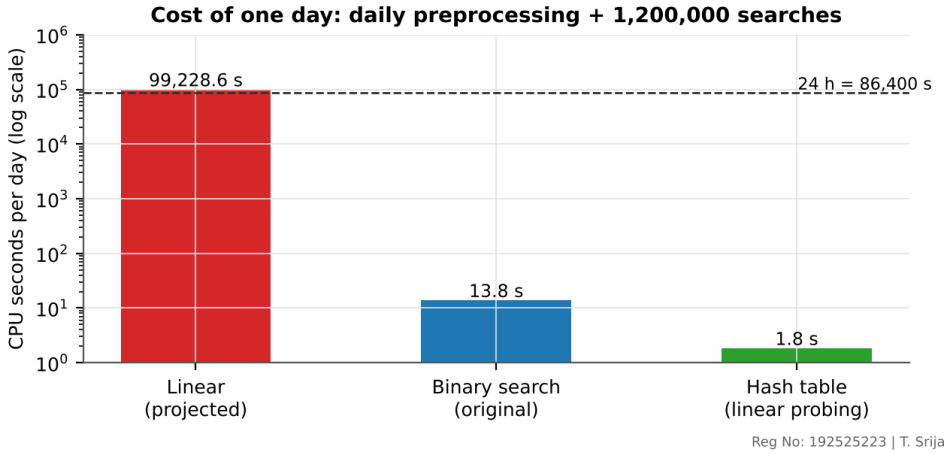


Figure 12.2: Cost of one day (daily preprocessing + 1,200,000 searches), log scale

Behaviour under 50,000 searches per hour. The required rate is only 13.9 searches per second. Binary search can serve about 89,557 per second on one core, i.e. it would be busy 0.016% of the time. Linear search manages only 12.1 searches per second on this machine, *less than the required 13.9*: it would need 115% of a core, so the queue would grow without bound, and one day of searching would cost 99,229 s — more than the 86,400 s of the day. Linear search is therefore not acceptable.

Is the preprocessing cost acceptable? Yes: sorting takes 0.40 s once a day. Building the hash table (0.47 s) is equally acceptable, but needs 44 MB instead of 8 MB. **Could another approach perform better?** Marginally: the hash table is the cheapest design per day (1.83 s of CPU against 13.80 s), because its faster searches (1.13 µs against 11.17 µs) outweigh the build over 1.2 million searches. The saving, however, is only 12.0 s out of 86,400 s per day.

C. Adaptation or Redesign (2 marks)

Decision: MODIFY — keep the search strategy (binary search on a sorted array, Θ(log n) per search) and change only how the sorted array is produced and published.

Aspect	Original design	Revised design
Search	recursive binary search	same algorithm; the iterative form may be used in production to avoid call overhead (same Θ(log n))
Preparing the data	re-sort the complete data set whenever it changes: Θ(n log n)	nightly batch job: sort the day's k new records (Θ(k log k)) and merge them with the existing sorted array (Θ(n + k))
During the day	records could change at any moment	the sorted array is an immutable snapshot, read-only for 24 hours; the new snapshot replaces the old one in a single step after the merge
Cost of the daily update	re-sort of 1,005,000 records: 0.433 s measured	merge of the sorted batch: 0.038 s measured (k = 5,000 new records; old run + new run, one linear merge)

Effect on complexity. Searching is unchanged: Θ(log n), about 20 comparisons for $n = 10^6$. The daily preparation improves from Θ(n log n) to Θ(n + k log k), where k is the number of new records; in this measurement merging the two sorted runs took 0.038 s against 0.433 s for a complete re-sort (11× faster; a pure-Python heapq.merge needed 0.193 s, rebuilding the hash table 0.558 s). Total cost of one day: Θ(n + k log k + q log n) with q = 1,200,000, i.e. about $10^6 + 6\times10^4 + 2.4\times10^7$ comparisons. In absolute terms the gain is small at $n = 10^6$ (0.39 s per day) but it is a real asymptotic gain that grows with n, and the merged array is verified to be identical to a full sort.

D. Evidence-Based Defence (2 marks)

Evidence	Finding	Consequence for the decision
Search frequency	13.9 searches/s = 1.2 million per day	linear search (only 12.1 searches/s, 115% of a core) is rejected; an indexed structure is needed
Dataset size	10^6 records; $\log_2 n \approx 20$	binary search needs about 19 comparisons per search
Update frequency	once per day	a static structure with a nightly merge is enough; no online insertion is needed
Preprocessing cost	sort 0.40 s, merge 0.038 s, hash build 0.47 s	the nightly merge is the cheapest preparation by far, about 15× cheaper than rebuilding the hash table
Capacity	binary 89,557/s against a need of 13.9/s	binary search has about 6,448× headroom; the hash table's extra speed adds no practical benefit
Memory	8 MB against 44 MB	binary search is much lighter
Functionality	sorted order	range queries (a block of registration numbers) remain possible; a hash table would lose them

Final decision. Retain binary search as the search algorithm and change the preprocessing to a nightly merge of the sorted batch that produces a read-only snapshot. This design serves 50,000 searches in 0.56 s (0.016% of the hour), prepares the daily data in about 0.04 s, needs 8 MB of extra memory and keeps range queries. The hash table remains the fastest option per search and saves 12.0 s of CPU per day, so it should be reconsidered only if the request rate rose by several orders of magnitude; with the given requirement the evidence does not justify its extra memory and loss of ordered queries.

Limitations of this study. All timings come from one machine and one run per size in Python; the linear-search time for 50,000 searches is projected from 100 measured searches, not measured in full; the number of new records per day (5,000) is an assumption. The conclusions rest on the ranking and the ratios of the measurements, which are supported by the exact comparison counts.